

Implementazione di code con priorità

Aggiunta di priorityqueue al package

- Arricchiamo il package con:
datastructure.priorityqueue
- Definiamo prima le interfaces **PriorityQueue.java** e **PriorityQueueNode.java**
 - **PriorityQueue** è l'interfaccia della struttura dati. Per avere accesso diretto ai nodi della struttura dati, usiamo l'interfaccia **PriorityQueueNode**
 - Un nodo conterrà una coppia <chiave,elemento> (più informazioni aggiuntive se utili per l'implementazione)
- Poi implementeremo le due interfacce usando l'approccio d-heap: **Dheap.java** e **DheapNode.java**

Interfaccia PriorityQueue

```
// K type of the key, D type of the data

public interface PriorityQueue<K,D> {

    public D findMin();
    // returns null if the queue is empty

    public PriorityQueueNode<K,D> insert(K key, D data);
    // returns the inserted node

    public void delete(PriorityQueueNode<K,D> node);
    // removes one node from the priority queue

    public void deleteMin();
    // removes the node with the minimum key

    public void increaseKey(K newKey, PriorityQueueNode<K,D> node);
    // no changes if K smaller

    public void decreaseKey(K newKey, PriorityQueueNode<K,D> node);
    // no changes if K bigger

    public boolean isEmpty();
}
```

Interfaccia PriorityQueueNode

```
// K type of the key, D type of the data  
  
public interface PriorityQueueNode<K,D> {  
  
    public K getKey(); // returns the key in the node  
  
    public D getData(); // returns the datum in the node  
  
}
```

Implementazione tramite d-heap

- Inseriamo i nodi del d-heap in un `ArrayList` seguendo i principi visti per i cosiddetti arrayheap
- Usiamo l'`ArrayList` invece di un array standard in quanto la dimensione del d-heap cambia dinamicamente (tramite operazioni **insert** e **delete**)
- Richiamo operazioni utili su **`ArrayList<E>`**

```
boolean add(E e) // Appends the specified element to the end of this list
E get(int index) // Returns the element at the specified position
E set(int index, E element) // Replaces the element at the specified
                           // position with the specified element
E remove(int index) // Removes the element at the specified position
```

Implementazione DHeapNode

```
// K type of the key, D type of the data

public class DHeapNode<K,D> implements PriorityQueueNode<K,D> {
    // key value
    protected K key;
    // data object
    protected D data;
    // node index (the index of the node in the array)
    protected int index;

    // Constructs a node containing the pair (key,data) with index i
    public DHeapNode(K key, D data, int i) {
        this.data = data; this.key = key; this.index = i;
    }
    // Returns the node key
    public K getKey() {
        return this.key;
    }
    // Returns the node datum
    public D getData() {
        return this.data;
    }
}
```

Implementazione DHeap

```
import java.util.ArrayList;

public class DHeap<K extends Comparable<K>,D> implements PriorityQueue<K,D> {

    private ArrayList<DHeapNode<K,D>> nodes; //ArrayList containing the nodes
    private int n; //Number of elements in the priority queue
    private int d; //Degree of the d-heap

    public Dheap(){ this(4); }

    public DHeap(int d){
        this.nodes = new ArrayList<DHeapNode<K,D>>(); this.n = 0; this.d = d;
    }

    public D findMin() { ... }

    public PriorityQueueNode<K,D> insert(K key, D data) { ... }

    public void delete(PriorityQueueNode<K,D> node) { ... }

    public void deleteMin() { ... }

    public void increaseKey(K newKey, PriorityQueueNode<K,D> node) { ... }

    public void decreaseKey(K newKey, PriorityQueueNode<K,D> node) { ... }

    public boolean isEmpty() { ... }
}
```

Metodi ausiliari utili

- Si forniscono già implementati alcuni metodi utili

```
private void moveUp(DHeapNode<K,D> v) {
    DHeapNode<K,D> f = father(v);
    while (v.index > 0 && (v.key).compareTo(f.key) < 0) {
        swap(v, f);
        f = father(v);
    }
}

private void moveDown(DHeapNode<K,D> v) {
    DHeapNode<K,D> min = minSon(v);
    while(min != null && (v.key).compareTo(min.key) > 0) {
        swap(v, min);
        min = minSon(v);
    }
}
```


Metodi ausiliari utili

```
private void swap(DHeapNode<K,D> n1, DHeapNode<K,D> n2) {
    nodes.set(n1.index,n2);
    nodes.set(n2.index,n1);
    int temp = n1.index;
    n1.index = n2.index;
    n2.index = temp;
}

private DHeapNode<K,D> father(DHeapNode<K,D> v) {
    if (v.index == 0) return null;
    return nodes.get( (v.index-1)/d );
}

private DHeapNode<K,D> minSon(DHeapNode<K,D> v) {
    int f = (d * (v.index)) + 1;
    if (f >= n) return null;
    DHeapNode<K,D> min = nodes.get(f);
    for (int i=f+1; (i < f+d) && (i < n); i++) {
        if ( ((nodes.get(i)).key).compareTo(min.key) < 0 )
            min = nodes.get(i);
    }
    return min;
}
```

Pseudocodice metodi pubblici

```
public D findMin() {  
    if (non ci sono elementi) return null  
    else return nodes.get(0).data  
}
```

```
public PriorityQueueNode<K,D> insert(K key, D data) {  
    node = new DheapNode<K,D>(key,data,n) //nuovo nodo va in posiz. n  
    nodes.add(node) //inserisce il nuovo nodo in fondo a nodes  
    n++ //incrementa il numero di nodi  
    moveUp(node) //sistema la posizione del nuovo nodo  
    return node  
}
```

Pseudocodice metodi pubblici

```
public void delete(PriorityQueueNode<K,D> node) {  
    u = nodes.remove(n-1) //estrae l'ultimo nodo e lo salva in u  
    n-- //decrementa il numero di nodi  
    if (u != node)  
        i = ((DHeapNode<K,D>) node).index //indice nodo da eliminare  
        nodes.set(i,u) //inserisce u al posto del nodo da eliminare  
        u.index = i //sistema il campo index del nodo spostato  
        moveUp(u) //sistema la posizione del nodo u  
        moveDown(u)  
}
```

```
public void deleteMin() {  
    if (ci sono elementi)  
        delete(nodes.get(0)) //usa delete per eliminare il primo nodo  
}
```

Pseudocodice metodi pubblici

```
public void increaseKey(K newKey, PriorityQueueNode<K,D> node) {  
    if (newKey è maggiore di node.getKey())  
        dnode = (DheapNode<K,D>) node //downcast di node  
        dnode.key = newKey //aggiornamento della chiave  
        moveDown(dnode) //sistema la posizione del nodo modificato  
}
```

```
public void decreaseKey(K newKey, PriorityQueueNode<K,D> node) {  
    if (newKey è minore di node.getKey())  
        dnode = (DheapNode<K,D>) node //downcast di node  
        dnode.key = newKey //aggiornamento della chiave  
        moveUp(dnode) //sistema la posizione del nodo modificato  
}
```

```
public boolean isEmpty() {  
    if (non ci sono elementi) return true  
    else return false  
}
```